Loan Risk and Transaction Analysis of a Czech Bank

Prepared by Karan Singh | Data Analyst | MySQL, Power BI

1. Summary

This project analyses 682 loans, 4,500 accounts and 1,056,320 transactions from a Czech bank (1993 to 1998) using MySQL and Power BI. The goal was to find out which factors separate loans that go bad from loans that are repaid, and how money moves through the bank over time.

Headline results

- The overall default rate is 11.1% (76 of 682 loans), where default means loan status B or D.
- The balance an account held before the loan was the strongest signal: the lowest-balance quarter of loans defaulted at 19.3%, about four times the highest-balance quarter (4.7%).
- Default rates differ by region, from 1.6% (North Bohemia) to 15.8% (West Bohemia).
- Defaulted loans were larger: B loans averaged 54% more than A loans, and D loans 45% more than C loans.
- Client age and district average salary did not separate risky from safe loans.
- Transaction volume grew 45% to 48% a year through 1997 and slowed to 13% in 1998, with strong seasonal spikes every January, June and December.
- Adding one index cut a slow query on the 1.06M-row transaction table from 4.3 seconds to 0.14 seconds (about 31 times faster).

2. Business questions

1. What is the overall loan default rate?
2. Does the balance an account held before the loan relate to default?
3. How does default vary by region?
4. Do loan amount, duration or monthly payment differ between good and bad loans?
5. Does the time between opening an account and taking a loan relate to default?

6. Does default differ by the year the loan was issued?
7. Does client age at the loan date matter?
8. Do higher-salary districts have lower default rates?
9. How do monthly transaction volume and value trend, and how seasonal are they?
10. Which payment purposes carry the most value, and what is inside the group with no purpose code?
11. Which withdrawals are unusually large for their own account?

3. Dataset and tools

The data is the Financial (Berka) dataset from the PKDD'99 Discovery Challenge, obtained from the CTU Prague Relational Dataset Repository. It is anonymised real data from a Czech bank, with amounts in koruna. It was exported from the public repository and loaded into a local MySQL 8.0 database. Dates run from 1993-01-01 to 1998-12-31, so all 72 months are complete.

Table	Contents
account	Bank accounts and the district where each was opened (4,500 rows)
client	Client gender, birth date and district (5,369 rows)
disp	Links clients to accounts as OWNER or DISPONENT
loan	Loans with amount, duration, monthly payment and status (682 rows)
order	Standing payment orders (6,471 rows)
trans	Every transaction with type, operation, amount, balance and purpose code (1,056,320 rows)
card	Cards issued, linked through disp
district	Regional demographics, including average salary

Clients connect to accounts, loans and cards only through the disp table, while loans, standing orders and transactions belong to accounts. Only account owners can take loans, so every loan analysis that uses client attributes joins the OWNER row only, which

also prevents loans from being counted twice.

Tools: MySQL 8.0 and MySQL Workbench for all analysis (joins, CTEs, window functions, indexing), and Power BI Desktop for the dashboard.

How the analysis was done

1. Loaded the public repository data into a local MySQL 8.0 database and confirmed row counts against the dataset description (4,500 accounts, 5,369 clients, 1,056,320 transactions).
2. Profiled every table with DESCRIBE, then checked keys, NULLs, duplicates and category values before writing any analysis.
3. Defined default as loan status B or D and used the 11.1% overall rate as the baseline for every comparison.
4. Answered each business question in SQL, moving from joins and aggregation to CTEs, NTILE and window functions.
5. Checked every result for sample size and timing effects (for example, loan age in 1998) before drawing a conclusion.
6. Measured one slow query with EXPLAIN, added an index, and measured again.
7. Published three SQL views (loan features, monthly totals, purpose totals) as the data source for the Power BI dashboard.

4. Data quality review

Before answering any business question, each table was checked for duplicate keys, missing values, unusual values and inconsistent labels.

Check	Finding	How it was handled
Duplicate primary keys	None found	No action needed
trans.operation missing	183,114 rows (17.3%)	Left as NULL. The count matches the number of interest-credit rows, which suggests these are system-generated entries

Check	Finding	How it was handled
trans.k_symbol (purpose code) missing	481,881 NULLs (45.6%) plus 53,433 empty strings (5.1%), 535,314 in total (50.7%)	Grouped as UNCATEGORIZED with COALESCE(NULLIF(TRIM(k_symbol), ''), 'UNCATEGORIZED')
order.k_symbol empty strings	1,379 of 6,471 standing orders (21%) stored as empty strings, not NULL	Found because a plain NULL check would have reported no missing values
Zero-amount transactions	162 rows (0.015%)	Kept, but excluded from per-account averages
Inconsistent type labels	Cash withdrawals appear under two type labels (VYDAJ and VYBER)	Both treated as withdrawals
Amount sign	Always positive; direction comes from type	Totals describe money moved, not net flow
Loan status	A 203, B 31, C 403, D 45	Default defined as B or D

Code reference

Meanings of the codes used in this report follow the dataset documentation.

Field	Value	Meaning
loan.status	A	Finished, repaid without problems
loan.status	B	Finished, not repaid (counted as default)
loan.status	C	Running, no problems so far
loan.status	D	Running, client in debt (counted as default)
trans.type	PRIJEM	Money in
trans.type	VYDAJ	Money out
trans.type	VYBER	Withdrawal (treated like VYDAJ cash withdrawals)

Field	Value	Meaning
trans.operation	VKLAD	Cash deposit
trans.operation	VYBER	Cash withdrawal
trans.operation	VYBER KARTOU	Card withdrawal
trans.operation	PREVOD NA UCET	Transfer out
trans.operation	PREVOD Z UCTU	Transfer in
k_symbol	SIPO	Household payment
k_symbol	DUCHOD	Pension
k_symbol	UVER	Loan payment
k_symbol	UROK	Interest credited
k_symbol	POJISTNE	Insurance payment
k_symbol	SLUZBY	Service fees
k_symbol	SANKC. UROK	Penalty interest
order.k_symbol	LEASING	Leasing payment

5. SQL techniques used

Technique	Where it was used
Multi-table JOINs	Loan to account to district, and loan to account to disp to client
CASE expressions and conditional counts	Default flags, age groups, waiting-time bands
CTEs	Balance and salary quartiles, monthly trend, large-withdrawal detection
NTILE	Splitting loans into four equal groups by balance and by salary
LAG	Month-over-month change in transactions and value

Technique	Where it was used
Window AVG with PARTITION BY	Each account's own average withdrawal
RANK	Ranking payment purposes by value
COALESCE and NULLIF	Handling both NULL and empty-string missing values
Subqueries	Average pre-loan balance per loan
Indexing and EXPLAIN	Query performance improvement

6. Findings: loan risk

6.1 Overall default rate

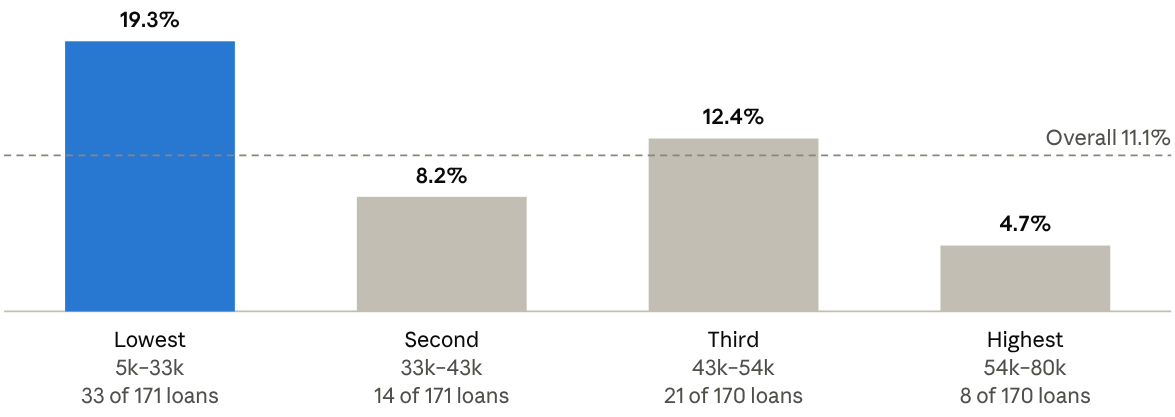
Of 682 loans, 76 have status B or D, a default rate of 11.1%. Counting status B alone (finished, unpaid) the rate is 4.5%. Status D means the loan is still running but the client is in debt, so it is closer to "at risk" than a final default.

6.2 Balance before the loan

Loans were split into four equal groups by the account's average balance before the loan date.

Lowest-balance accounts default at 19.3%, four times the highest (4.7%)

Default rate by average account balance before the loan (koruna); 682 loans in four equal groups



SQL on loan and trans (balance averaged before the loan date) · 682 loans

Balance group	Average balance before loan	Loans	Defaults	Default rate
Lowest	5,350 to 32,788	171	33	19.3%
Second	32,968 to 43,197	171	14	8.2%
Third	43,197 to 53,636	170	21	12.4%
Highest	53,686 to 79,501	170	8	4.7%

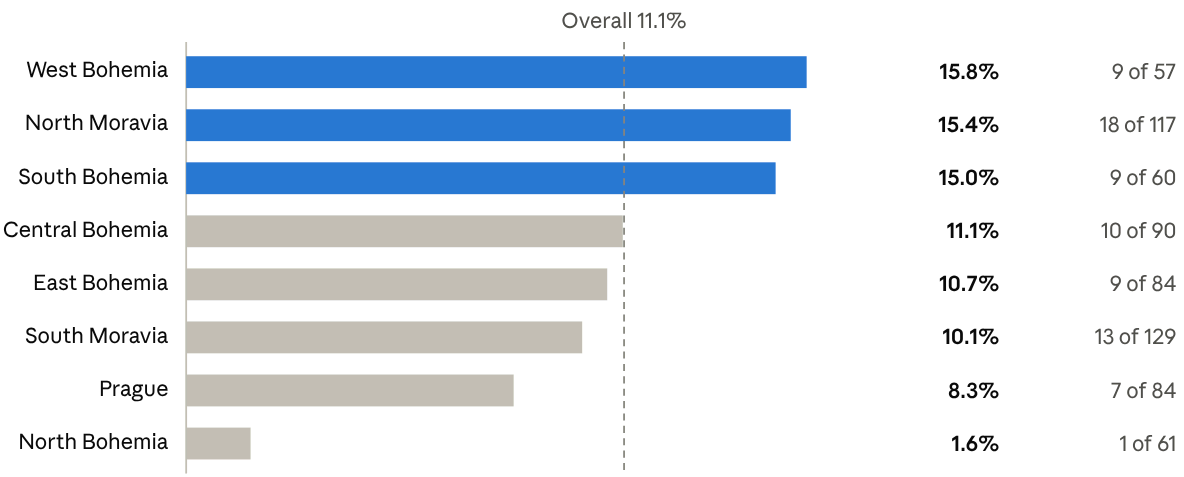
The lowest group defaulted about four times as often as the highest. The trend is not perfectly smooth (the third group is above the second), so the safe conclusion is that low-balance accounts are clearly riskier than high-balance ones.

6.3 Region

Region is the region of the district where the account was opened.

Three regions default above 15%, while North Bohemia is at 1.6%

Default rate by region of the district where the account was opened; right column shows defaults of loans



SQL on loan, account and district · 682 loans, 8 regions

Region	Loans	Defaults	Default rate
West Bohemia	57	9	15.8%
North Moravia	117	18	15.4%
South Bohemia	60	9	15.0%

Region	Loans	Defaults	Default rate
Central Bohemia	90	10	11.1%
East Bohemia	84	9	10.7%
South Moravia	129	13	10.1%
Prague	84	7	8.3%
North Bohemia	61	1	1.6%

North Moravia has the most defaults (18 of 117 loans). North Bohemia had only one default in 61 loans, which is unusual but still a small count. District-level rates were not used for conclusions because most districts have fewer than 10 loans.

6.4 Loan size, duration and payment

Status	Loans	Average amount	Average duration (months)	Average monthly payment
A	203	91,641	22.2	4,264
B	31	140,721	25.6	5,396
C	403	171,410	43.4	3,939
D	45	249,285	46.1	5,287

Comparing A with B, and C with D, the defaulted loans are larger (54% and 45% higher average amount) and carry higher monthly payments (27% and 34% higher). Duration differs by only about three months within each pair, so loan size and payment burden matter more than loan length.

6.5 Time between opening the account and taking the loan

No loan was taken sooner than about 3.5 months after the account was opened.

Time to loan	Loans	Defaults	Default rate
3 to 6 months	82	14	17.1%
6 to 12 months	216	21	9.7%

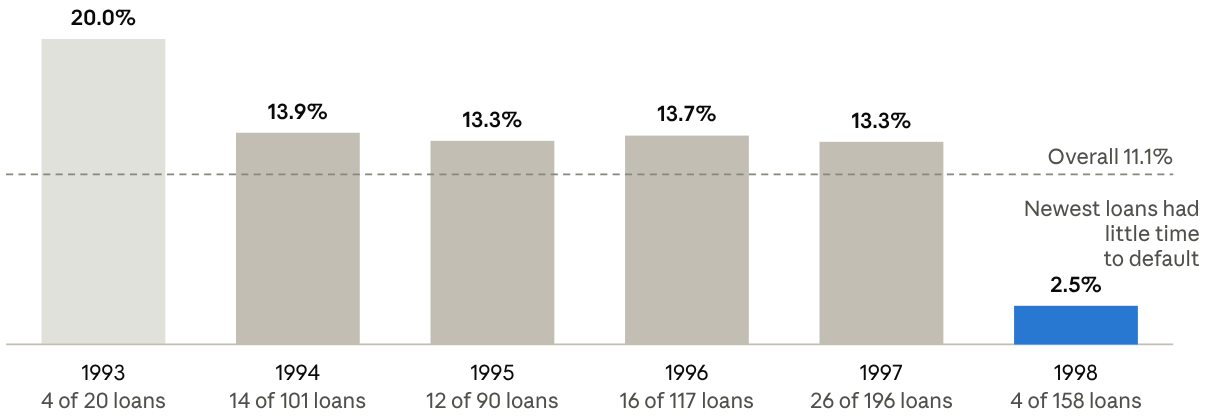
Time to loan	Loans	Defaults	Default rate
12 to 18 months	229	28	12.2%
18+ months	155	13	8.4%

Borrowing 3 to 6 months after opening an account is associated with the highest default rate, but the pattern is not steady across groups and the first group is small, so this is a possible warning signal rather than a firm finding.

6.6 Year the loan was issued

Default held near 13% for 1994 to 1997; 1998 looks low because its loans are new

Default rate by year the loan was issued; the faded bar has fewer than 30 loans



SQL on loan, account and district · 682 loans by year of issue

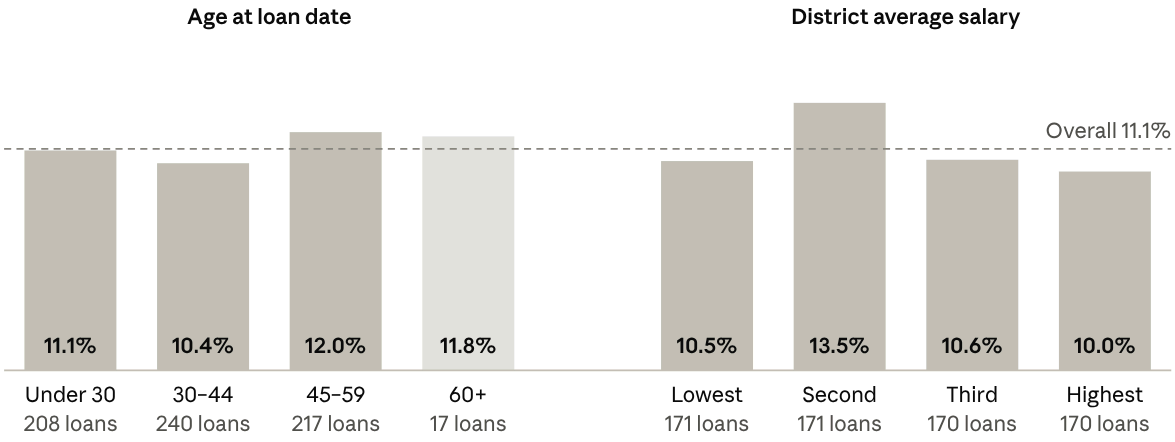
Year	Loans	Defaults	Default rate
1993	20	4	20.0%
1994	101	14	13.9%
1995	90	12	13.3%
1996	117	16	13.7%
1997	196	26	13.3%
1998	158	4	2.5%

Rates were steady at about 13% to 14% for 1994 to 1997. The drop to 2.5% for 1998 most likely reflects loan age, because the newest loans had little time to default, so it should not be read as an improvement. Excluding 1998, the default rate is about 13.7%, so the overall 11.1% probably understates the long-run rate.

6.7 Client age and district salary

Neither client age nor district salary moves default far from 11.1%

Default rate by client age and by district average salary (four equal groups); the faded bar has fewer than 30 loans



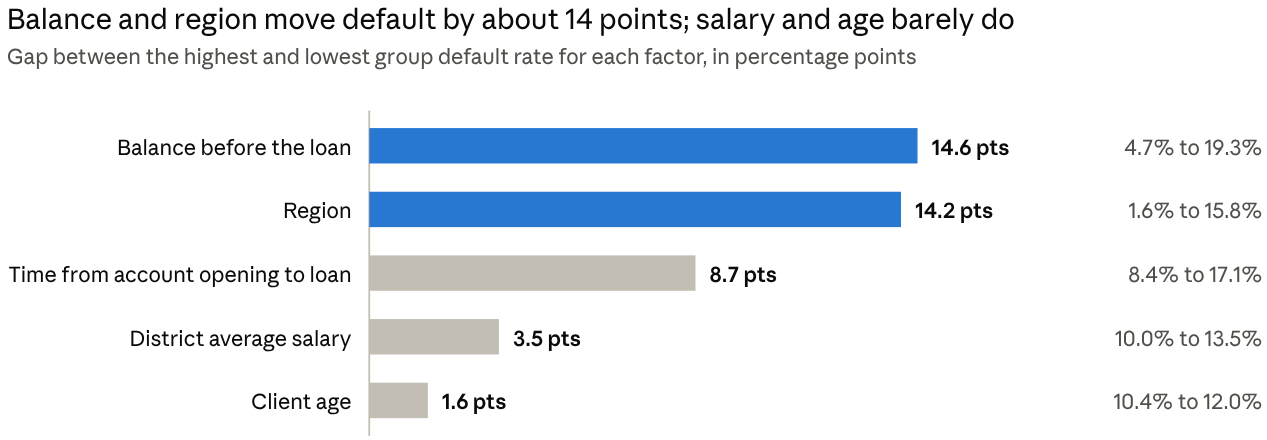
SQL on loan, disp, client, account and district · age uses account owners only · 682 loans in each panel

Age at loan date	Loans	Defaults	Default rate
Under 30	208	23	11.1%
30 to 44	240	25	10.4%
45 to 59	217	26	12.0%
60+	17	2	11.8%

District average salary	Loans	Defaults	Default rate
8,110 to 8,544	171	18	10.5%
8,544 to 8,991	171	23	13.5%
8,991 to 9,897	170	18	10.6%
9,897 to 12,541	170	17	10.0%

Neither factor shows a meaningful pattern. All age groups sit close to the overall rate, and the lowest and highest salary groups both default at about 10%. The 60+ group has only 17 loans and should not be interpreted.

6.8 Which factors matter most



Gaps computed from the group default rates in sections 6.2, 6.3, 6.5 and 6.7

Factor	Gap between highest and lowest group
Balance before the loan	14.6 percentage points
Region	14.2 points
Time to loan	8.7 points
District salary	3.5 points
Client age	1.6 points

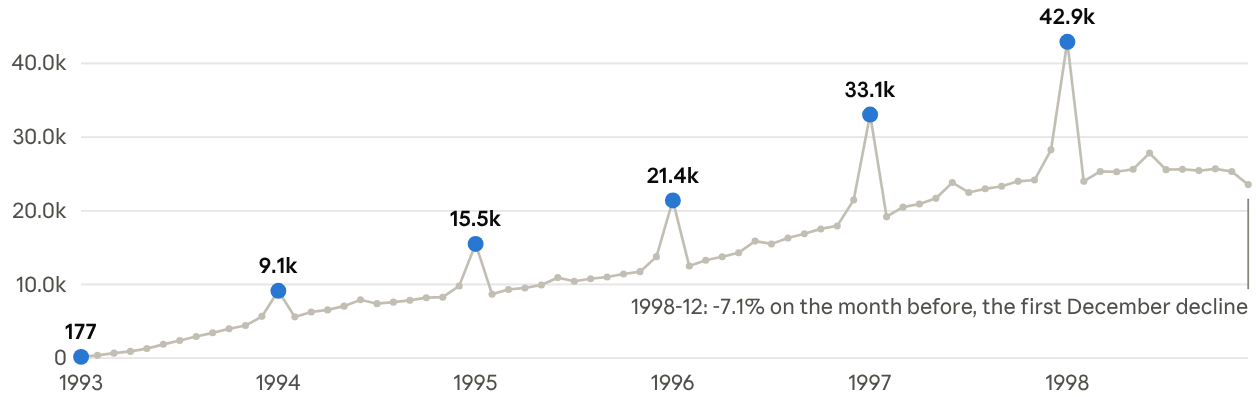
The client's own financial behaviour (balance) separates risk far better than the wealth of their district or their age. Year of issue is left out of this comparison because of the 1998 effect described above.

7. Findings: transactions

7.1 Growth

Monthly transactions grew from 177 to about 25,000, with a spike every January

1,056,320 transactions over 72 months; dots mark each January



SQL: monthly COUNT(*) on trans · 72 months, 1993-01 to 1998-12

Year	Transactions	Growth on previous year
1993	28,205	n/a
1994	91,628	+225% (ramp-up from a very small base)
1995	133,022	+45%
1996	196,779	+48%
1997	284,409	+45%
1998	322,277	+13%

Monthly volume rose from 177 transactions in January 1993 to about 25,000 a month in 1998. Growth held at 45% to 48% a year through 1997 and then slowed to 13%.

7.2 Seasonality

Month	Pattern (1994 to 1998 unless noted)
January	Volume jumps 52% to 61%, but value rises only about 5% to 8% (many small transactions)
February	Volume falls 39% to 44%
June	Value jumps 36% to 43% on a rise in transaction count of only about 9% to 12% (fewer, larger transactions)
July	Value falls 24% to 30%
December	Value rises 28% to 31% in 1994 to 1997; in 1998 volume fell 7.1%, the first December decline

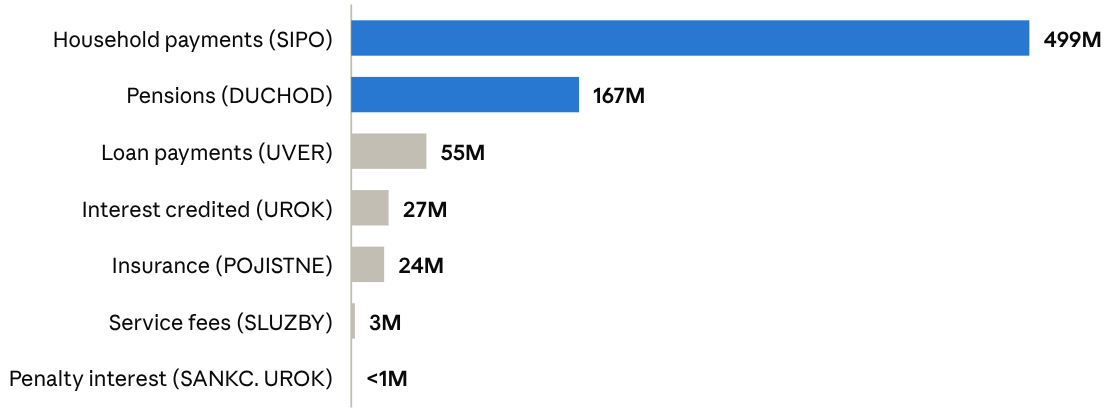
The data covers the full month of December 1998, so that decline is a real slowdown and not a partial month.

7.3 Payment purposes

Purpose is the k_symbol code on each transaction. Code meanings follow the dataset documentation.

Household payments and pensions carry 86% of the money moved under a purpose code

No purpose code: 5,482M (87.6% of all value) is not shown; values in millions of koruna



SQL on trans grouped by k_symbol · millions of koruna · code meanings follow the dataset documentation

Purpose	Transactions	Value (millions)	Share of total value
No purpose code	535,314	5,482	87.6%
SIPO (household payments)	118,065	499	8.0%
DUCHOD (pension)	30,338	167	2.7%
UVER (loan payments)	13,580	55	0.9%
UROK (interest credited)	183,114	27	0.4%
POJISTNE (insurance)	18,500	24	0.4%
SLUZBY (service fees)	155,832	3	under 0.1%
SANKC. UROK (penalty interest)	1,577	under 1	under 0.1%

Among labelled purposes, household payments carry the most value, followed by pensions and loan payments. Frequent purposes such as interest credits and service fees carry almost no value (average transaction about 150 and 18 koruna).

7.4 What the uncategorised transactions are

The 535,314 transactions without a purpose code are ordinary cash and transfer activity, so the missing code reflects the type of transaction and not damaged data.

Operation	Type	Transactions	Share of the group
Cash withdrawal (VYBER)	VYDAJ	258,009	48.2%
Cash deposit (VKLAD)	PRIJEM	156,743	29.3%
Transfer out (PREVOD NA UCET)	VYDAJ	60,972	11.4%
Transfer in (PREVOD Z UCTU)	PRIJEM	34,888	6.5%
Cash withdrawal (VYBER)	VYBER	16,666	3.1%
Card withdrawal (VYBER KARTOU)	VYDAJ	8,036	1.5%

7.5 Unusually large withdrawals

A withdrawal was flagged when it was more than five times that account's own average withdrawal, using accounts with at least ten withdrawals. This compares each customer with their own habits instead of with the whole bank.

- 7,551 withdrawals (about 1.2%) across 2,606 accounts (58% of all accounts) were flagged.
- The most extreme were 18.7 to 27.5 times the account average (amounts of 31,680 to 68,600).
- Some accounts were flagged repeatedly (one account four times), which points to regular large payments and not one-off anomalies.
- Withdrawals of 30,000 or more were most common in June (1,904), then December (1,200), matching the months with the biggest jumps in transaction value.

Because more than half of all accounts were flagged, the output works as a review list and not as a fraud detector. The five-times threshold is a judgement call and can be tuned.

8. Query performance

A query summing UVER (loan payment) transactions per account was slow because the only useful index, on account_id, forced MySQL to read about 1.04 million index entries and look up each row to apply the k_symbol filter.

	Before	After index on k_symbol
Run time	4.344 seconds	0.141 seconds (about 31 times faster)
EXPLAIN access type	index (full index scan)	ref (direct lookup)
Rows examined (estimate)	1,037,965	25,044

The trade-off is extra disk space and slightly slower inserts, which is acceptable for a read-heavy analytics table. The new plan also uses a temporary table for the GROUP BY, which is cheap on about 25,000 rows.

9. Dashboard

A Power BI dashboard presents the results. It connects to three SQL views: a loan-level view (balance and salary quartiles, age groups, waiting-time bands, default flag), a monthly transaction view and a payment-purpose view. Measures include Loans, Defaulted Loans, Default Rate, Overall Default Rate, Average Loan Amount, Transactions and Money Moved.

[Insert dashboard screenshots here]

10. Recommendations

These are suggestions based on associations in this sample, not proven causes.

1. **Use pre-loan balance as a screening signal.** Applicants whose accounts held the least money before borrowing carried the highest default rate (19.3%).
2. **Review lending in higher-default regions** such as North Moravia and West Bohemia (about 15% to 16%), and investigate why North Bohemia is so low before copying its practices.
3. **Watch early borrowers.** Loans taken 3 to 6 months after opening an account defaulted at 17.1% and could be monitored more closely.
4. **Add affordability checks on loan size and monthly payment**, since defaulted loans were consistently larger with heavier payments.
5. **Do not rely on age or district average salary alone**, because neither separated risky from safe loans.
6. **Plan capacity for seasonal peaks** in January (volume), and June and December (value).
7. **Improve data capture:** standardise the type label used for cash withdrawals and record purpose codes consistently, so missing codes are easier to interpret.

11. Limitations and assumptions

- Default is defined as loan status B or D. Status meanings (A finished and repaid, B finished and unpaid, C running without problems, D running with the client in debt) follow the dataset documentation.
- Loans from 1998 are too recent to judge, so recent default rates are understated.
- Several groups are small (for example 17 loans aged 60+ and 20 loans issued in 1993), so differences of a point or two are within chance.

- Region is where the account was opened, not necessarily where the client lives. District salary is a district average (column A11 in the dataset documentation), not the client's own income.
- Purpose and operation names are translated from the dataset codes.
- Thresholds such as five times the account average, and the age and waiting-time bands, are analyst choices.
- All findings are associations, not causes. The data is from the 1990s and amounts are in koruna, so it describes relationships in this sample and not current market conditions.

12. Conclusion

In this bank's data, the strongest predictor of loan default was how much money an account held before the loan, followed by region and loan size, while client age and district salary added little. The transaction history shows a fast-growing, strongly seasonal bank whose growth slowed in 1998. The project also shows how careful data checks change conclusions: a plain NULL check would have missed 53,433 empty purpose codes, and the 1998 default rate looks good only because those loans had not had time to go wrong.

Appendix: sample SQL

Default rate by balance quartile (CTE and NTILE)

```
WITH pre_loan AS (
    SELECT l.loan_id, l.status, AVG(t.balance) AS avg_balance,
           NTILE(4) OVER (ORDER BY AVG(t.balance)) AS balance_quartile
    FROM loan l
    JOIN trans t ON t.account_id = l.account_id AND t.date < l.date
    GROUP BY l.loan_id, l.status
)
SELECT balance_quartile, COUNT(*) AS loans,
       SUM(status IN ('B','D')) AS defaulted_loans,
       ROUND(AVG(status IN ('B','D')) * 100, 2) AS default_rate_pct
FROM pre_loan
GROUP BY balance_quartile
ORDER BY balance_quartile;
```

Index that cut the query from 4.3 seconds to 0.14 seconds

```
CREATE INDEX idx_trans_ksymbol ON trans(k_symbol);
```

Overall default rate and default rate by region

```
SELECT COUNT(*) AS total_loans,
       SUM(status IN ('B','D')) AS defaulted_loans,
       ROUND(AVG(status IN ('B','D')) * 100, 2) AS default_rate_pct
FROM loan;

SELECT d.A3 AS region, COUNT(*) AS loans,
       SUM(l.status IN ('B','D')) AS defaulted_loans,
       ROUND(AVG(l.status IN ('B','D')) * 100, 2) AS default_rate_pct
FROM loan l
JOIN account a ON a.account_id = l.account_id
JOIN district d ON d.district_id = a.district_id
GROUP BY d.A3
ORDER BY default_rate_pct DESC;
```

Default rate by age group (account owners only)

```
SELECT CASE WHEN TIMESTAMPDIFF(YEAR, c.birth_date, l.date) < 30 THEN 'Under
30'
            WHEN TIMESTAMPDIFF(YEAR, c.birth_date, l.date) < 45 THEN '30-44'
            WHEN TIMESTAMPDIFF(YEAR, c.birth_date, l.date) < 60 THEN '45-59'
            ELSE '60+' END AS age_group,
       COUNT(*) AS total_loans,
       SUM(l.status IN ('B','D')) AS defaulted_loans,
       ROUND(AVG(l.status IN ('B','D')) * 100, 2) AS default_rate_pct
FROM loan l
JOIN disp d ON d.account_id = l.account_id AND d.type = 'OWNER'
JOIN client c ON c.client_id = d.client_id
GROUP BY age_group
ORDER BY MIN(TIMESTAMPDIFF(YEAR, c.birth_date, l.date));
```

Time from opening the account to the loan

```
SELECT CASE WHEN DATEDIFF(l.date, a.date) < 180 THEN '3-6 months'
            WHEN DATEDIFF(l.date, a.date) < 365 THEN '6-12 months'
            WHEN DATEDIFF(l.date, a.date) < 545 THEN '12-18 months'
            ELSE '18+ months' END AS account_age_at_loan,
       COUNT(*) AS total_loans,
       SUM(l.status IN ('B','D')) AS defaulted_loans,
       ROUND(AVG(l.status IN ('B','D')) * 100, 2) AS default_rate_pct
FROM loan l
JOIN account a ON a.account_id = l.account_id
GROUP BY account_age_at_loan
ORDER BY MIN(DATEDIFF(l.date, a.date));
```

Monthly volume and month-over-month change (CTE and LAG)

```
WITH monthly AS (
    SELECT DATE_FORMAT(date, '%Y-%m') AS month,
           COUNT(*) AS trans_count,
           SUM(amount) AS total_amount
    FROM trans
    GROUP BY DATE_FORMAT(date, '%Y-%m')
),
with_prev AS (
    SELECT month, trans_count, total_amount,
           LAG(trans_count)  OVER (ORDER BY month) AS prev_count,
           LAG(total_amount) OVER (ORDER BY month) AS prev_amount
    FROM monthly
)
SELECT month, trans_count, total_amount,
       ROUND((trans_count - prev_count) * 100 / prev_count, 2)   AS
volume_mom_pct,
       ROUND((total_amount - prev_amount) * 100 / prev_amount, 2) AS
value_mom_pct
FROM with_prev
ORDER BY month;
```

Unusually large withdrawals (window function)

```
WITH withdrawals AS (
    SELECT account_id, trans_id, date, amount,
           AVG(amount) OVER (PARTITION BY account_id) AS account_avg,
           COUNT(*) OVER (PARTITION BY account_id) AS account_withdrawals
    FROM trans
    WHERE type IN ('VYDAJ', 'VYBER') AND amount > 0
)
SELECT account_id, trans_id, date, amount,
       ROUND(account_avg) AS account_avg_withdrawal,
       ROUND(amount / account_avg, 1) AS times_above_avg
FROM withdrawals
WHERE amount > 5 * account_avg AND account_withdrawals >= 10
ORDER BY times_above_avg DESC
LIMIT 20;
```

Payment purposes (COALESCE, NULLIF, RANK)

```
SELECT COALESCE(NULLIF(TRIM(k_symbol), ''), 'UNCATEGORIZED') AS purpose,
       COUNT(*) AS trans_count,
       ROUND(AVG(amount), 2) AS avg_amount,
       SUM(amount) AS total_amount,
       RANK() OVER (ORDER BY SUM(amount) DESC) AS rank_by_value
FROM trans
GROUP BY COALESCE(NULLIF(TRIM(k_symbol), ''), 'UNCATEGORIZED')
ORDER BY rank_by_value;
```

Index test: the query measured before and after, and its plan

```
SELECT account_id, SUM(amount) AS total_loan_payments
FROM trans
WHERE k_symbol = 'UVER'
GROUP BY account_id;

EXPLAIN SELECT account_id, SUM(amount) AS total_loan_payments
FROM trans
WHERE k_symbol = 'UVER'
GROUP BY account_id;
```

SQL views used as the Power BI data source

```
CREATE OR REPLACE VIEW v_loans AS
SELECT x.*,
       NTILE(4) OVER (ORDER BY avg_balance)     AS balance_quartile,
       NTILE(4) OVER (ORDER BY district_salary) AS salary_quartile,
       CASE WHEN age_at_loan < 30 THEN 'Under 30'
            WHEN age_at_loan < 45 THEN '30-44'
            WHEN age_at_loan < 60 THEN '45-59'
            ELSE '60+' END AS age_group,
       CASE WHEN days_to_loan < 180 THEN '3-6 months'
            WHEN days_to_loan < 365 THEN '6-12 months'
            WHEN days_to_loan < 545 THEN '12-18 months'
            ELSE '18+ months' END AS wait_band
FROM (
    SELECT l.loan_id, l.account_id, l.date AS loan_date, YEAR(l.date) AS
loan_year,
           l.amount, l.duration, l.payments, l.status,
           CASE WHEN l.status IN ('B','D') THEN 1 ELSE 0 END AS is_default,
           d.A3  AS region,
           d.A11 AS district_salary,
           DATEDIFF(l.date, a.date) AS days_to_loan,
           TIMESTAMPDIFF(YEAR, c.birth_date, l.date) AS age_at_loan,
           pb.avg_balance
    FROM loan l
    JOIN account a  ON a.account_id = l.account_id
    JOIN district d ON d.district_id = a.district_id
    JOIN disp dp    ON dp.account_id = l.account_id AND dp.type = 'OWNER'
    JOIN client c   ON c.client_id = dp.client_id
    LEFT JOIN (
        SELECT l2.loan_id, AVG(t.balance) AS avg_balance
        FROM loan l2
        JOIN trans t ON t.account_id = l2.account_id AND t.date < l2.date
        GROUP BY l2.loan_id
    ) pb ON pb.loan_id = l.loan_id
) x;

CREATE OR REPLACE VIEW v_monthly AS
SELECT DATE_FORMAT(date, '%Y-%m-01') AS month_start,
       COUNT(*) AS trans_count,
       SUM(amount) AS total_amount
FROM trans
GROUP BY DATE_FORMAT(date, '%Y-%m-01');

CREATE OR REPLACE VIEW v_purpose AS
```

```
SELECT COALESCE(NULLIF(TRIM(k_symbol), ''), 'UNCATEGORIZED') AS purpose,
       COUNT(*) AS trans_count,
       SUM(amount) AS total_amount
FROM trans
GROUP BY COALESCE(NULLIF(TRIM(k_symbol), ''), 'UNCATEGORIZED');
```